the same task (e.g., weather forecasting or translation), and generate balanced, provider-agnostic user queries that all APIs in a cluster can answer. The final benchmark consists of 10 clusters, each containing 5 APIs and 100 queries, yielding 1,000 total cluster-query pairs. Running LLMs on this benchmark produces empirical API-selection distributions, from which we compute our bias metrics δ_{API}, δ_{pos}, and δ_{model}. See Appendix A for clustering and query-generation details.

3.3 Explaining Bias

To pinpoint what drives tool-selection bias, we pursue three complementary analyses.

Attribute-Level Analysis. To test whether intrinsic API characteristics explain model preferences, we extract seven descriptive features for each API. Examples include semantic similarity between queries and descriptions, number of parameters, description length, readability, and promotional wording (See Table 3 for the full list). Our benchmark contains 10 clusters with 5 APIs each, giving 50 APIs in total. For each API, we measure its empirical selection rate and pair this with its feature values. This produces a dataset of 50 (API, features, selection rate) entries per model. We then probe relationships between features and selection behavior in three ways. First, we compute *Pearson correlations* between tool features and selection rate to capture linear and monotonic associations. Second, we fit a *linear regression* per model to quantify the aggregate explanatory power (reported as R^2) and inspect coefficients to understand the direction and relative weight of each feature. Third, we train *random-forest regressors* with cross-validation to allow for non-linear interactions and obtain alternative measures of feature importance. This pipeline reveals which API attributes most strongly influence the model's choices.

Metadata Perturbation Experiments. To isolate the superficial cues that drive tool-selection preferences, we apply a series of controlled perturbations to tool metadata. Specifically, we utilize the following manipulations: (1) *Full name scramble.* Replace every tool's name with a fresh 20-character random string, destroying any learned association tied to the literal name; (2) *Name shuffle.* Permute the tool names among APIs within each cluster so that names no longer align with their original endpoints; (3) *Single-tool perturbation.* Identify the most frequently chosen tool in each cluster and replace only its name with a random string; (4) *Description and parameter scramble.* Randomize each tool's descriptive text and parameter description(s) (but keep the original names intact) to test whether the semantic content beyond the name influences selection; (5) *Description-only / Parameter-only scramble.* Randomize only the tool descriptions (keeping parameter descriptions intact), or only the parameter descriptions (keeping the tool description intact), to disambiguate their individual contributions; (6) *Targeted description scramble (most-selected).* Identify the most frequently chosen API in each cluster and scramble only *its* description to test whether degrading its semantics reduces its selection share; (7) *Description transfer (most → least).* Swap the most-selected API's description with the least-selected API while leaving other metadata untouched, assessing whether swapping semantic "advantage" transfers selection probability; and (8) *Full scramble.* Randomize each tool's descriptive text, parameter description(s), and tool names to test the effect of having minimal semantic signal in API metadata on bias.

By re-running our selection experiments under these alterations, we quantify how much of the observed bias is attributable to literal names, to deeper semantic content in descriptions and parameters, and to relative contrast between a clean and corrupted endpoint.

Biased Continued Pre-Training. We also test whether pre-training data itself can induce tool-selection bias. To verify this hypothesis, we perform biased continued pre-training (CPT) on Qwen3-8B using $\sim$3.5M tokens deliberately saturated with a single API's metadata. See Appendix C for additional details on corpus construction and training setup.

4 Experiments

This section reports the empirical results from our experiments. We first describe the experimental setup (Section 4.1), then characterize tool-selection behavior and quantify bias using our metrics (Section 4.2). Next, we investigate the drivers of bias (Section 4.3), and finally, we evaluate an approach to mitigate any observed bias in Section 4.4.

```
Available tools:
  * API A
  * API B
  * API C

User:
What is the weather in Paris?
```

```
Available tools:
  * API B
  * API C
  * API A

User:
What is the weather in Paris?
```

```
Available tools:
  * API C
  * API A
  * API B

User:
What is the weather in Paris?
```

Figure 2: Cyclic rotations of one fixed tool list; each API appears at the top once.

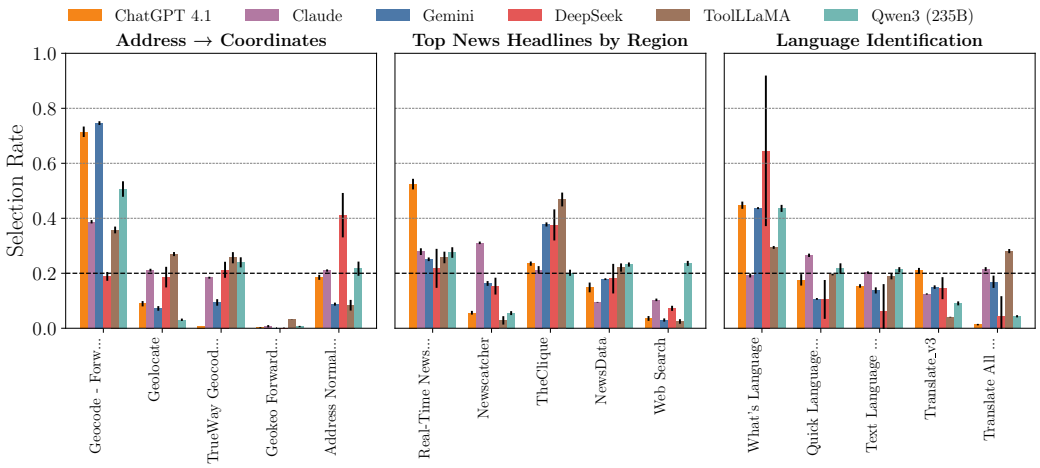

Figure 3: Selection distributions for six LLMs across three clusters of functionally equivalent APIs. Each subplot corresponds to one cluster, with the x-axis indicating the API in the cluster and the y-axis showing the (mean) fraction of times each model chose that API over 500 inference runs; error bars indicate the standard deviation across three independent experimental runs. The optimal uniform selection rate is highlighted.

4.1 Experimental set-up

Models. We evaluate a diverse set of LLMs, including ChatGPT-3.5-turbo and 4.1-mini by OpenAI (2025), Claude 3.5 Sonnet by Anthropic (2025), DeepSeek R1 (DeepSeek-AI et al., 2025), Gemini 2.5 Flash (Google, 2023), ToolLLaMA-2-7B (Qin et al., 2024), and Qwen3 models spanning 1.7B to 235B parameters (Bai et al., 2023).

Reasoning. Each model is prompted to produce a short chain of thought (Wei et al., 2022) followed by at most one tool call. This design makes outputs efficient to generate and easy to analyze.

Parameter Setup & Dataset. Experiments use our benchmark of 10 clusters, each containing 5 interchangeable APIs and 100 distinct user queries (Section 3.2 and Appendix A). Unless otherwise noted, decoding uses a temperature of 0.5 and a top-p of 1.0.

API ordering. As noted in previous work (Faghih et al. (2025)), LLMs prefer tools that appear earlier in the prompt. To control for this, we execute each query five times, each with a different cyclic rotation of a fixed API ordering (see Figure 2). This ensures that every API appears at the top for a given query exactly once.

4.2 How do LLMs select among functionally equivalent APIs?

Figure 3 shows our first results: the empirical selection distributions of six LLMs over three clusters of functionally equivalent APIs (see Appendix G.2 for the full figure including all clusters). Choices are far from uniform. In some clusters (e.g., geocoding), all models concentrate heavily on a single API, whereas in others (e.g., language identification) the distributions are flatter. We also observe

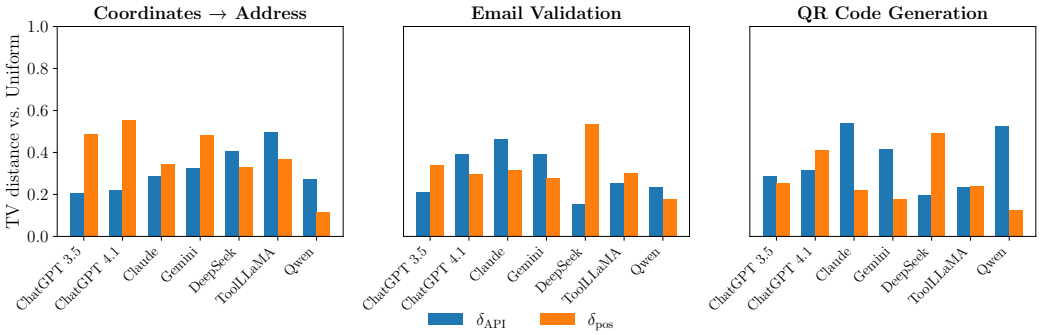

Figure 4: API- vs. positional bias by model for three clusters. Bars show total-variation deviation from uniform, where higher values indicate stronger bias.

Table 1: Average cluster-level API bias δ_{API}, positional bias δ_{pos}, and combined bias δ_{model} (mean across clusters and runs $\pm$ std across runs).

Model	δ_{API}	δ_{pos}	δ_{model}
Gemini 2.5 Flash	$0.365_{\pm.003}$	$0.306_{\pm.005}$	$0.335_{\pm.002}$
Claude 3.5 Sonnet	$0.370_{\pm.005}$	$0.325_{\pm.005}$	$0.347_{\pm.001}$
DeepSeek R1	$0.292_{\pm.043}$	$0.443_{\pm.054}$	$0.367_{\pm.006}$
Qwen3 235B	$0.330_{\pm.006}$	$0.168_{\pm.004}$	$0.249_{\pm.001}$
ToolLLaMA	$0.277_{\pm.002}$	$0.391_{\pm.002}$	$0.334_{\pm.002}$
ChatGPT 3.5-turbo	$0.320_{\pm.022}$	$0.336_{\pm.012}$	$0.328_{\pm.005}$
ChatGPT 4.1-mini	$0.331_{\pm.008}$	$0.423_{\pm.002}$	$0.377_{\pm.004}$

that different models do not always prefer the same API, a point we explore further when analyzing alignment in selection behavior.

Positional bias persists even when API-level preference is weak. Figure 4 summarizes API- vs. positional-bias (measured as total variation distance from uniform) across models and clusters. We notice two regimes: high API bias with low positional bias, or low API bias with high positional bias; a few exhibit both elevated. This pattern indicates that when no API clearly dominates, models rely more on positional cues.

The extent of LLM bias in tool selection. To get an idea of *how* biased each model is in their tool-selection behavior, we compute the positional bias δ_{pos}, API bias δ_{API}, and their average δ_{model} for seven different models (Section 3.1). Table 1 reports these metrics averaged across all clusters and three runs. All models exhibit substantial bias: δ_{model} values are around 0.3–0.4, meaning that roughly 30–40% of the selection probability mass would have to be redistributed to achieve fairness. ChatGPT 4.1-mini is especially biased, with a combined metric of $\sim$0.38. The least biased model in our suite is Qwen 3 (235B), which attains the lowest δ_{model}.

LLMs are (mostly) aligned in their bias. To examine the similarity of selection behavior of the different models, we represent each model by a vector obtained by concatenating its empirical selection distributions over APIs across all clusters, then compute pairwise Pearson correlations between these vectors; the resulting matrix is shown in Figure 15. It shows that many of the models share similar bias patterns: GPT 4.1-mini, Claude, Gemini, DeepSeek, and Qwen3 (235B) tend to favor and disfavor the same APIs. GPT-3.5 and ToolLLaMA stand apart with consistently lower correlations, suggesting qualitatively different selection behavior. This clustering of high correlations points to common drivers of bias. This could be due to a shared set of similar implicit decision rules. The relative divergence of GPT-3.5 and ToolLLaMA highlights that model architecture, capacity, or